

SMART_AO V8 — Spécification d'implémentation des commandes normalisées et de l'idempotence

Version : 1.0

Statut : spécification technique à appliquer avant l'écriture des premières commandes d' API

Auteur : Manus AI

Périmètre : commandes d'écriture du noyau patron, concurrence, idempotence, faits métier, outbox et actualisation des situations préparées.

1. Objectif

Une action patron n' est jamais un simple bouton qui modifie des colonnes. Elle devient une **commande normalisée** : une demande explicite, authentifiée, contextualisée, protégée contre les répétitions et soumise aux invariants de la frontière métier propriétaire.

Cette spécification garantit notamment qu' un double clic, une reconnexion réseau, un rafraîchissement de navigateur ou une reprise d' appel API ne peut pas créer deux validations de prix, deux décisions Go/No-Go, deux paquets de dépôt ou deux affectations concurrentes.

Règle absolue : la réponse « succès » n' est renvoyée que lorsque le changement métier, ses faits métier et son résultat idempotent ont été validés ensemble dans une transaction durable.

2. Chaîne d' exécution obligatoire

Plain Text

Interface autorisée

- Commande normalisée
 - Réserve de clé d'idempotence
 - Chargement contrôlé de la frontière métier
 - Préconditions et invariants
 - Transition durable
 - Faits métier + outbox

→ Résultat de commande mémorisé
→ Projection Cockpit / autres situations préparées

La projection de Cockpit ne décide jamais. Elle affiche les conséquences des faits métier acceptés. Une panne de projection ne remet donc pas en cause la validation de la décision, du prix ou du dépôt ; elle rend seulement la vue temporairement en actualisation ou partielle .

3. Enveloppe d' une commande normalisée

3.1. Données contrôlées par le serveur

L' interface cliente ne fournit jamais tenant_id , actor_id , rôle ou permissions comme données fiables. Ces informations sont dérivées de la session authentifiée côté serveur.

Champ	Origine	Règle
command_id	Client ou serveur	UUID unique de l' intention. Recommandé : UUID généré côté client avant l' envoi.
command_type	Client contrôlé	Nom fermé d' une commande autorisée.
idempotency_key	Client	UUID ou chaîne opaque stable, unique pour une intention de mutation.
correlation_id	Client ou serveur	Identifie un parcours métier commun : par exemple création d' affaire puis import DCE.
causation_id	Serveur	Commande ayant causé le fait métier. Par défaut : command_id .
expected_versions	Client	Versions lues par l' utilisateur pour les ressources sensibles.
decision_context_hash	Client quand requis	Empreinte du contexte effectivement affiché au patron.
payload	Client	Données validées par le schéma de la commande concernée.

<code>tenant_id</code>	Session serveur	Toujours extrait du contexte authentifié.
<code>actor_id</code> et <code>actor_role</code>	Session serveur	Toujours extraits de la session et revérifiés par la politique d' autorisation.
<code>issued_at</code>	Serveur	Horodatage du traitement, pas heure librement déclarée par le navigateur.

3.2. Modèle Python de référence

Python

```

from __future__ import annotations

from datetime import datetime
from typing import Any, Literal
from uuid import UUID

from pydantic import BaseModel, Field

class ExpectedVersion(BaseModel):
    boundary_type: Literal[
        "AFF", "DCE", "ACT", "DEC", "PRF", "PRX", "DEP", "ASN", "ORG", "OPP"
    ]
    boundary_id: UUID
    version: int = Field(ge=0)

class CommandEnvelope(BaseModel):
    command_id: UUID
    command_type: str
    idempotency_key: UUID
    correlation_id: UUID | None = None
    expected_versions: list[ExpectedVersion] = []
    decision_context_hash: str | None = None
    payload: dict[str, Any]

class AuthenticatedCommandContext(BaseModel):
    tenant_id: UUID
    actor_id: UUID

```

```
actor_role: Literal["PATRON", "DELEGATAIRE", "COLLABORATEUR", "PARTENAIRE"]
received_at: datetime
```

Les modèles Pydantic concrets doivent être spécialisés par commande. Une charge utile non conforme est rejetée avant toute réservation métier, avec un message explicable au client.

4. Catalogue initial des commandes

Domaine	Commandes initiales	Frontière propriétaire
Affaire	CreateAffair , AssignAffair , ReassignAffair , MoveAffairToDecision , MoveAffairToPricing , StopAffair , ArchiveAffair	AFF
DCE	RegisterDceVersion , ClassifyDceDocument , MarkDceImpactReviewed	DCE
Action patron	AcknowledgePatronAction , DelegateActionPreparation , SetActionConditions , CompletePatronAction , AbandonPatronAction , SupersedePatronAction	ACT
Décision	PrepareDecision , ApproveDecision , ApproveDecisionWithConditions , RejectDecision , SupersedeDecision	DEC
Entreprise / preuve	AddCapability , ConfirmCapabilityForAffair , AddEvidenceVersion , AuthorizeEvidenceForAffair , ArchiveEvidence	ORG ou PRF
Affectation	InviteCollaborator , AssignCollaboratorToAffair , RemoveCollaboratorFromAffair , SuspendAccount , GrantTemporaryDelegation	ASN

Opportunité	CreateOpportunityProfile , QualifyOpportunity , TransmitOpportunity , ConvertOpportunityToAffair , DiscardOpportunity	OPP
Prix privé	CreatePricingScenario , UpdatePricingScenario , PrepareOfficialPricingVersion , ApprovePricingVersion , MarkPricingVersionStale	PRX
Dépôt	PrepareSubmissionPackage , AuthorizeSubmission , DeclareSubmission , ArchiveSubmissionReceipt , SupersedeSubmissionPackage	DEP

Le catalogue est fermé par défaut. Une commande non listée est rejetée. Toute nouvelle commande doit être ajoutée à la matrice V8, au Contrat de domaine, au schéma Pydantic, à la politique d' autorisation et à la suite de tests.

5. Contrat d' idempotence

5.1. Clé et empreinte

Une même intention doit toujours utiliser la même clé d' idempotence. La clé est unique dans le périmètre suivant :

Plain Text

(tenant_id, actor_id, command_type, idempotency_key)

La commande calcule aussi une empreinte SHA-256 d' une représentation JSON canonique des éléments suivants :

Plain Text

command_type
payload
expected_versions triées
decision_context_hash

Le `tenant_id`, l'acteur et les autorisations ne sont pas fournis dans l'empreinte client : ils sont ajoutés et contrôlés côté serveur.

Situation	Réponse imposée
Première réception de la clé	La commande obtient une réservation et poursuit le traitement.
Même clé, même empreinte, résultat réussi	Retourner exactement le résultat mémorisé, sans nouvelle mutation ni nouvel événement.
Même clé, même empreinte, traitement encore en cours	Retourner <code>202 IDEMPOTENCY_IN_PROGRESS</code> , avec une URL/identifiant de suivi et un délai de réessai conseillé.
Même clé, empreinte différente	Retourner <code>409 IDEMPOTENCY_KEY_REUSED</code> ; aucun traitement n'est exécuté.
Même clé, rejet métier déjà mémorisé	Rejouer le même rejet. L'utilisateur doit relire la situation et émettre une nouvelle commande avec une nouvelle clé.
Clé absente ou invalide pour une commande critique	Rejeter avant toute mutation avec <code>400 IDEMPOTENCY_KEY_REQUIRED</code> .

5.2. Registre durable des commandes

Le registre d'idempotence est une donnée de production, pas un cache mémoire. Il doit survivre au redémarrage du conteneur et être sauvegardé avec PostgreSQL.

Colonne conceptuelle	Rôle
<code>id</code>	Identifiant interne du registre.
<code>tenant_id</code> , <code>actor_id</code> , <code>command_type</code> , <code>idempotency_key</code>	Clé unique de l'intention.
<code>request_hash</code>	Empreinte canonique de la commande.
<code>status</code>	<code>PROCESSING</code> , <code>SUCCEEDED</code> , <code>REJECTED</code> , <code>FAILED_RETRYABLE</code> , <code>EXPIRED</code> .
<code>lease_expires_at</code>	Expiration contrôlée d'un traitement interrompu avant résultat.
<code>aggregate_refs</code>	Frontières touchées et versions finales.

<code>http_status</code> , <code>result_code</code> , <code>response_body</code>	Résultat rejouable, sans secret ni donnée non autorisée.
<code>event_ids</code>	Faits métier acceptés par la commande.
<code>created_at</code> , <code>completed_at</code>	Audit et nettoyage contrôlé.

La contrainte unique PostgreSQL est obligatoire sur :

SQL

```
UNIQUE (tenant_id, actor_id, command_type, idempotency_key )
```

Les contraintes d'unicité et `INSERT ... ON CONFLICT` permettent de réserver une intention face à des appels concurrents ; `ON CONFLICT DO UPDATE` fournit un résultat atomique d'insertion ou de mise à jour hors erreur indépendante. ¹

5.3. Durées de conservation

Catégorie de commande	Conservation minimale recommandée	Motif
Décision, prix, dépôt, affectation, document	365 jours	Actions engageantes, audit et reprises réseau tardives.
Création ou modification non critique	90 jours	Relecture et diagnostic raisonnables.
Commande échouée techniquement	30 jours	Analyse d'incident ; aucune donnée métier n'est confirmée.
Données de réponse contenant un secret	0 jour	Les secrets ne doivent jamais être mémorisés dans le registre.

La purge est un processus administré : elle ne supprime jamais les faits métier, décisions, versions ou preuves ; elle ne concerne que le registre de rejouabilité après la durée de conservation définie.

6. Algorithme de traitement

6.1. Étapes obligatoires

1. Authentifier la session et construire le `AuthenticatedCommandContext` côté serveur.
2. Valider le schéma Pydantic de la commande et la politique de rôle.
3. Construire l'empreinte canonique de la requête.
4. Réserver ou relire l'intention dans le registre d'idempotence.
5. Si un résultat terminal existe avec la même empreinte, le retourner immédiatement.
6. Ouvrir une transaction courte pour la mutation métier.
7. Charger les frontières nécessaires dans un ordre stable ; verrouiller seulement les ressources nécessaires.
8. Vérifier tenant, droits, versions attendues, contexte de décision et invariants métier.
9. Produire la transition durable, les faits métier et les messages d'outbox.
10. Mémoriser le résultat de succès dans le même commit que la mutation et les faits métier.
11. Après commit, laisser les projecteurs actualiser Cockpit et autres situations préparées.
12. Retourner le résultat durable ; le client peut l'afficher sans attendre toutes les projections.

6.2. Pseudo-code de référence

Python

```
async def dispatch_command(
    envelope: CommandEnvelope,
    auth: AuthenticatedCommandContext,
) -> CommandResponse:
    spec = command_registry.get(envelope.command_type)
    spec.validate_payload(envelope.payload)
    authorize(auth, spec.required_permission, envelope.payload)

    request_hash = canonical_hash(envelope)
    receipt = await idempotency_store.reserve_or_fetch(
        tenant_id=auth.tenant_id,
        actor_id=auth.actor_id,
        command_type=envelope.command_type,
        key=envelope.idempotency_key,
        request_hash=request_hash,
    )

    if receipt.is_hash_mismatch:
        raise Conflict("IDEMPOTENCY_KEY_REUSED")
    if receipt.is_terminal:
```



```

        return receipt.replay_response()
    if receipt.is_processing_by_other_worker:
        return CommandResponse.processing(receipt.status_url)

    try:
        async with unit_of_work.transaction(spec.isolation_level):
            handler = spec.handler
            outcome = await handler.execute(
                envelope=envelope,
                auth=auth,
                unit_of_work=unit_of_work,
            )
            await domain_event_store.append(outcome.events)
            await outbox.enqueue(outcome.events)
            await idempotency_store.mark_succeeded(
                receipt_id=receipt.id,
                response=outcome.response,
                aggregate_refs=outcome.aggregate_refs,
                event_ids=outcome.event_ids,
            )
            return outcome.response
    except BusinessRuleViolation as error:
        await idempotency_store.mark_rejected(
            receipt_id=receipt.id,
            error=error.to_client_error(),
        )
        raise
    except RetryableConcurrencyError:
        await idempotency_store.release_or_mark_retryable(receipt.id)
        raise

```

Aucun appel SMTP, OCR, LLM, navigateur de profil acheteur ou service partenaire ne doit être exécuté dans la transaction métier. Ces effets externes passent par l’ outbox après commit.

7. Contrôle de concurrence et versions attendues

7.1. Deux protections complémentaires

Protection	Usage	Exemple
Version attendue	Empêche l’ écrasement d’ une ressource lue avant une modification concurrente.	Le patron valide un prix lu en version 7 alors qu’ un rectificatif l’ a déjà rendu version 8.

Contexte figé	Empêche une décision sur une synthèse devenue différente, même si la frontière décision n' a pas changé.	Go/No-Go préparé avant l' arrivée d' un rectificatif DCE ou d' un devis remplacé.
----------------------	--	---

Une mutation simple utilise une clause de version optimiste. Une mutation critique portant sur plusieurs frontières utilise une transaction courte et un chargement verrouillé selon l' ordre ci-dessous.

Plain Text

ORG → AFF → DCE → PRF → PRX → DEP → ACT → DEC → ASN → OPP

Cet ordre unique réduit le risque d' interblocage lorsque plusieurs ressources doivent être protégées. PostgreSQL recommande d' acquérir les verrous dans un ordre cohérent, de garder les transactions courtes et de réessayer proprement les transactions interrompues par un interblocage. ⁴

7.2. Règle d' écriture optimiste

Le stockage de chaque frontière possède un entier `version` . Une mise à jour compare cette valeur :

SQL

```
UPDATE pricing_versions
SET status = :new_status,
    version = version + 1,
    updated_at = NOW()
WHERE tenant_id = :tenant_id
    AND id = :pricing_version_id
    AND version = :expected_version;
```

Si aucune ligne n' est modifiée, le serveur relit la ressource et retourne :

JSON

```
{
  "code": "VERSION_CONFLICT",
  "message": "Le prix a été modifié ou rendu à revoir depuis votre dernière lec",
  "next_action": "Actualiser le dossier et reprendre votre décision.",
  "current_version": 8
}
```

7.3. Transactions renforcées

Les commandes `ApprovePricingVersion` , `AuthorizeSubmission` , `ArchiveSubmissionReceipt` , `ApproveDecision` , `ApproveDecisionWithConditions` et `RegisterDceVersion` peuvent mobiliser plusieurs frontières. Elles utilisent une transaction à isolation renforcée lorsque les invariants ne peuvent pas être garantis par une version locale et une contrainte simple.

PostgreSQL indique que les transactions `SERIALIZABLE` peuvent échouer avec `SQLSTATE 40001` et que l' application doit être préparée à reprendre la transaction complète. ² V8 applique au maximum trois réessais techniques courts, avec un délai aléatoire borné. Au-delà, le client reçoit `409 CONCURRENT_CHANGE_RETRY_REQUIRED` , sans faux succès.

8. Préconditions, invariants et postconditions

Niveau	Question	Exemple prix	Exemple dépôt
Précondition	« Puis-je examiner cette intention ? »	Patron authentifié ; version de prix accessible.	Patron/déléguataire habilité ; paquet existant.
Invariant	« Qu' est-ce qui doit rester vrai ? »	Un scénario ne modifie pas une version officielle.	Un paquet autorisé n' est plus modifiable.
Transition	« Quel changement métier est demandé ? »	<code>PREPAREE → VALIDEE_PATRON</code> .	<code>PRET_CONTROLE → AUTORISE_DEPOT</code> .
Postcondition	« Qu' est-ce qui doit être vrai après commit ? »	Une version officielle active existe avec snapshot des hypothèses.	Manifest du paquet, version DCE et version prix sont figés.
Fait métier	« Qu' est-ce qui est journalisé ? »	<code>PricingVersionApproved</code> .	<code>SubmissionAuthorized</code> .
Projection	« Quelles vues doivent évoluer ? »	Cockpit, affaire, action patron, prix privé, journal.	Cockpit, affaire, coffre, journal, actions.

Les contraintes SQL assurent les règles locales de structure et d' unicité ; les règles faisant intervenir plusieurs frontières restent dans le handler transactionnel. PostgreSQL précise qu' une contrainte `CHECK` ne doit pas chercher à assurer une condition qui dépend d' autres lignes ou tables. ³

9. Contrats détaillés de commandes sensibles

9.1. ApprovePricingVersion

Élément	Contrat
Frontière propriétaire	PRX
Autorité	Patron administrateur ou délégation explicite de validation de prix.
Préconditions	Affaire active ; version DCE applicable ; version de prix PREPAREE ou A_REVOIR ; prix non déjà remplacé ; contexte de décision courant affiché.
Versions attendues	PRX , AFF , DCE et, si utilisée, la version de devis/partenaire concernée.
Invariants	Aucun poste non couvert ou hypothèse fragile ne peut être caché ; le patron voit les éléments ouverts ; aucun collaborateur n' est bénéficiaire du résultat.
Transition	Version officielle PREPAREE/A_REVOIR → VALIDEE_PATRON .
Postconditions	Snapshot des pièces, hypothèses, calculs et devis ; action patron prix terminée ou remplacée ; ancienne version conservée.
Faits	PricingVersionApproved , éventuellement PatronActionCompleted .
Situations mises à jour	Cockpit, Vue de direction, Prix privé, Coffre de dépôt, Journal de vérité.
Refus typiques	STALE_CONTEXT , VERSION_CONFLICT , PRICING_NOT_READY , FORBIDDEN .

9.2. ApproveDecisionWithConditions

Élément	Contrat
Frontière propriétaire	DEC

Autorité	Patron administrateur uniquement, sauf délégation décisionnelle explicite et limitée.
Préconditions	Dossier de décision disponible ; conditions non vides ; chaque condition a responsable et date ou motif d'absence ; contexte valide.
Transition	EN_ATTENTE_PATRON → VALIDEE_SOUS_CONDITIONS .
Postconditions	Contexte figé ; conditions créées/rattachées ; actions de suivi créées sans doublon ; affaire passe seulement à l'état compatible.
Faits	DecisionApprovedWithConditions , PatronActionConditionSet , éventuellement AffairProgressed .
Situations mises à jour	Cockpit, Action Queue, Vue de direction, Portefeuille, Journal.

9.3. AuthorizeSubmission

Élément	Contrat
Frontière propriétaire	DEP
Autorité	Patron ou délégataire habilité au dépôt.
Préconditions	Paquet PRET_CONTROLE ; version officielle de prix validée ; version DCE applicable ; aucun blocage actif ; manifest de fichiers intègre les versions contrôlées.
Contexte requis	Empreinte de manifest, version DCE, version prix, état des contrôles et signature si le RC l'exige.
Transition	PRET_CONTROLE → AUTORISE_DEPOT .
Postconditions	Paquet immuable ; toute modification ultérieure impose un paquet remplaçant ; action dépôt devient prête à exécuter humainement.
Faits	SubmissionAuthorized .

Situations mises à jour	Cockpit, Coffre de dépôt, Affaire, Journal.
Interdit	Ne produit jamais <code>Dépôt réussi</code> .

9.4. ArchiveSubmissionReceipt

Élément	Contrat
Frontière propriétaire	<code>DEP</code>
Préconditions	Paquet autorisé ou déclaré déposé ; preuve fournie ; plateforme et date/heure renseignées ou extractibles ; lien vers l' affaire.
Transition	<code>DEPOT_DECLARE</code> → <code>ACCUSE_ARCHIVE</code> ou création directe de la preuve si le processus humain est documenté.
Postconditions	Accusé immuable associé à une version précise de paquet ; affaire peut afficher le dépôt prouvé.
Faits	<code>SubmissionReceiptArchived</code> .
Refus	Preuve incompatible, paquet inconnu, ressource inter-entreprise, double accusé contradictoire sans revue.

10. Faits métier, outbox et projecteurs

10.1. Écriture atomique

Dans une même transaction, le handler doit écrire :

1. la mutation de la frontière métier ;
2. le ou les faits métier ;
3. la ligne outbox pour chaque fait à projeter ou diffuser ;
4. le résultat `SUCCEEDED` de l' intention idempotente.

Cette règle évite le problème du double écriture : un prix validé sans événement ou un

événement envoyé alors que le prix a été annulé.

10.2. Outbox

Champ	Rôle
<code>outbox_id</code>	Identifiant immuable.
<code>event_id</code>	Référence au fait métier.
<code>topic</code>	Domaine cible, par exemple <code>cockpit_projection</code> .
<code>payload_version</code>	Contrat de charge utile.
<code>status</code>	<code>PENDING</code> , <code>PROCESSING</code> , <code>PUBLISHED</code> , <code>RETRY</code> , <code>FAILED</code> .
<code>attempt_count</code> , <code>next_attempt_at</code>	Gestion de réessai.
<code>dedupe_key</code>	Empêche un projecteur de traiter deux fois le même fait.

10.3. Projecteurs idempotents

Chaque projecteur maintient un marqueur (`projector_name`, `event_id`) unique. Si un message est livré deux fois, le second passage ne modifie plus la projection. Le Cockpit peut donc être reconstruit à partir des faits sans dupliquer une action ou une ligne de journal.

11. Contrat des réponses API

11.1. Succès

JSON

```
{
  "status": "SUCCEEDED",
  "command_id": "a2db5e12-65a4-4e0b-b039-3b062df31aa4",
  "idempotency_key": "2c12d459-bdb9-4d26-80bc-2626e4a4b8ff",
  "result_code": "PRICING_VERSION_APPROVED",
  "aggregate_refs": [
    {"type": "PRX", "id": "...", "version": 8},
    {"type": "AFF", "id": "...", "version": 13}
  ],
}
```

```

    "event_ids": ["..."],
    "next_navigation": {
      "view": "AFFAIR_EXECUTIVE",
      "affair_id": "..."
    },
    "projection_status": "REFRESH_PENDING"
  }

```

11.2. Rejet métier

JSON

```

{
  "status": "REJECTED",
  "code": "STALE_CONTEXT",
  "message": "Un rectificatif DCE a été reçu depuis la préparation de votre déc.",
  "next_action": "Relire les éléments modifiés et créer une nouvelle décision."
  "changed_resources": [
    {"type": "DCE", "id": "...", "current_version": 3}
  ]
}

```

11.3. Codes normalisés

Code	Sens pour l' interface
VALIDATION_ERROR	Champ ou format incorrect ; corriger avant nouvel envoi.
FORBIDDEN	Droits insuffisants ; ne pas exposer la ressource.
NOT_FOUND	Ressource inaccessible ou inexistante ; réponse neutre en cas d' isolement entreprise.
IDEMPOTENCY_KEY_REQUIRED	La commande critique doit porter une clé.
IDEMPOTENCY_KEY_REUSED	Même clé utilisée pour un autre contenu ; générer une nouvelle intention.
IDEMPOTENCY_IN_PROGRESS	Une exécution est en cours ; attendre le résultat plutôt que renvoyer.
VERSION_CONFLICT	La ressource a changé depuis la lecture ; actualiser.

STALE_CONTEXT	Une décision est basée sur un contexte devenu obsolète ; relire et créer une nouvelle intention.
BUSINESS_RULE_VIOLATION	Invariant métier non respecté ; le message indique l' action de correction.
RETRYABLE_CONCURRENCY_CONFLICT	Conflit technique temporaire ; le client peut réessayer selon le protocole.
PROJECTION_PENDING	Mutation validée, vues en cours d' actualisation ; ne pas soumettre à nouveau.

12. Sécurité, audit et confidentialité

Sujet	Exigence
Autorisation	Vérification avant chargement des données détaillées et avant toute mutation.
Journal	Toute commande réussie ou rejetée porte un acteur, une entreprise, un identifiant et un résultat ; la charge sensible est minimisée.
Prix privé	Les réponses de commande et les registres idempotents ne contiennent pas de montants détaillés si le client ou le destinataire n' y est pas autorisé.
Secrets	Aucun mot de passe, token API, clé de stockage, document binaire ou donnée bancaire complète dans événement, outbox ou réponse mémorisée.
Partage externe	Toute commande de demande partenaire référence un périmètre autorisé, une affaire et une date d' expiration.
Retrait d' accès	Une commande de suspension doit empêcher immédiatement les nouvelles commandes et lectures tout en préservant les traces historiques.

13. Tests obligatoires avant la première commande de production

Niveau	Test minimal
Unitaire	Chaque handler accepte les préconditions valides et rejette chaque invariant violé.
Propriété	Un scénario ne modifie jamais une version officielle ; une décision approuvée reste immuable.
Base de données	La contrainte unique d' idempotence et les clés étrangères empêchent les doublons et relations orphelines.
Concurrence	Deux <code>ApprovePricingVersion</code> simultanés n' aboutissent qu' à un seul succès durable ; l' autre reçoit un conflit ou rejoue le même résultat.
Réseau	Une requête interrompue après commit puis rejouée retourne le résultat mémorisé sans seconde mutation.
Crash	Une panne avant commit ne produit aucun fait métier ; une panne après commit est récupérable par la clé d' idempotence.
Outbox	Un même fait livré deux fois ne duplique ni action Cockpit ni ligne Journal.
Confidentialité	Un collaborateur ne peut pas obtenir, par commande, erreur, réponse mémorisée ou projection, un prix privé.
Rectificatif	L' arrivée d' un nouveau DCE marque les dépendances à revoir sans écraser une ancienne décision ou un paquet déjà déposé.

14. Décisions de gel

1. Toutes les écritures métier V8 passent par l' enveloppe de commande normalisée.

2. `tenant_id` , identité de l'acteur et permissions sont exclusivement déterminés côté serveur.
 3. La clé d'idempotence est obligatoire pour toute commande créant une décision, un prix, un paquet, un dépôt, une affectation ou une version de document.
 4. Les mutations critiques écrivent frontière, fait métier, outbox et résultat idempotent dans une transaction cohérente.
 5. Le Cockpit et les autres vues sont des projections idempotentes ; elles ne deviennent jamais la source de vérité.
 6. Une commande qui rencontre un contexte obsolète est rejetée explicitement ; elle ne tente aucune fusion silencieuse.
 7. Aucun effet externe n'est exécuté avant le commit métier.
-

Références

- [1] PostgreSQL — INSERT et ON CONFLICT
- [2] PostgreSQL — Niveaux d'isolation des transactions
- [3] PostgreSQL — Contraintes
- [4] PostgreSQL — Verrous explicites

Références internes

- `SMART_AO_V8_CONTRAT_DE_DOMAINE.md`
 - `SMART_AO_V8_CONTRAT_METIER_VERS_INTERFACE.md`
 - `SMART_AO_V8_MATRICE_TRANSITIONS_METIER.md`
 - `SMART_AO_V8_CAHIER_CHARGES_COCKPIT_PATRON.md`
 - `recherche_idempotence_postgres_v8.md`
-

Fin de la Spécification commandes normalisées et idempotence V8 — version 1.0